The Authorization-Execution Gap Is a Major Safety and Security Problem in Open-World Agents

Baoyuan Wu[1]* **Qingshan Liu**[2] **Adel Bibi**[3] **Irwin King**[4] **Siwei Lyu**[5]
[1]Chinese University of Hong Kong, Shenzhen, China
[2]Nanjing University of Posts and Telecommunications, China
[3]University of Oxford, UK
[4]Chinese University of Hong Kong, China
[5]State University of New York at Buffalo, USA
wubaoyuan@cuhk.edu.cn; qsliu@nuist.edu.cn; adel.bibi@eng.ox.ac.uk;
king@cse.cuhk.edu.hk; siweilyu@buffalo.edu

Abstract

This position paper argues that the **Authorization-Execution Gap (AEG)** is a major safety and security problem in open-world agents. The AEG is the divergence between what a principal intends to authorize and what an open-world agent ultimately executes. Because such agents act autonomously across tools, persistent state, and multi-agent handoffs, even small instances of authorization divergence can cause harm that is difficult or impossible to undo. We argue that many observed agent failures can be traced to three structural sources of AEG: delegation-level incompleteness, channel-level corruption, and composition-level fragmentation. The same observed failure may arise from any of these sources. Without identifying the source, a defense targeting the symptom alone cannot address the underlying cause. Agent safety and security should therefore emphasize source-oriented diagnosis and defense. Because the structural sources of AEG arise dynamically during execution, this approach necessarily requires authorization integrity checks applied during execution, rather than relying solely on one-shot upfront filtering or post-hoc audit. For NeurIPS, the implication is that papers on open-world agents should report not only outcome-level metrics such as task success or attack resistance, but also process-level evidence showing where AEG was detected, constrained, and attributed to a structural source during execution.

1 Introduction

1.1 The Fundamental Shift: From Output Generation to Delegated Execution

In large language models (LLMs), the human remains the executor: the model advises, the human decides and acts. In agentic systems, principals delegate bounded mandates to agents, which then decide and act autonomously across sequences of steps that may be costly or impossible to reverse. This shift is fundamental, not a matter of degree. This paper is therefore not about generic LLM safety in text-only settings, where the model produces outputs for a human to evaluate. It is about the distinct safety and security problem that arises when the model becomes an actor: it can use tools, write persistent state, and delegate work across time or across other agents.

Our position is that the Authorization-Execution Gap (AEG) is a major safety and security problem in open-world agents. The AEG is the divergence between what a principal intends to authorize and what an open-world agent ultimately executes. Many observed agent failures can

*Corresponding Author

Preprint.

be traced to three structural sources of AEG: delegation-level incompleteness, channel-level corruption, and composition-level fragmentation. Agent safety and security should therefore emphasize source-oriented diagnosis and defense, together with authorization integrity checks during execution.

Principals do not authorize isolated outputs; they authorize bounded actions in a workflow. That authorization is conveyed through delegated context and then carried into execution, where tool outputs, state updates, and downstream handoffs may themselves become decision-guiding. The central question is whether realized execution stays within the authorization scope established by delegation. AEG appears along this path when execution diverges from what the principal intended to authorize under pressure from execution-time inputs, persistent state, and multi-agent composition.

1.2 Structural Sources of the Gap and Why It Is Dangerous

AEG is hard to avoid in open-world agents because it arises from three structural sources. **First,** the principal's intended authorization must be conveyed through delegated context, which serves only as an incomplete operational proxy. As context is often expressed partly in natural language, boundary cases, exceptions, escalation conditions, and stopping rules may remain implicit or ambiguous. **Second,** execution channels are corruptible: tool outputs, web content, and stored state can redirect later steps when environmental content is treated as if it carried delegated authority. **Third,** authorization can fragment across stages, tools, and agents, because locally acceptable steps do not by themselves guarantee end-to-end authorization coherence.

Why autonomous execution makes the gap dangerous. In single-turn LLMs, an authorization gap usually produces incorrect text: the human can reject it, reinterpret it, or simply do nothing. In agentic systems, the same gap produces wrong *actions*, persistent state changes, or downstream handoffs that may have already taken effect before a human notices. Two execution properties amplify the gap into harm:

- **Irreversibility**: undoing costs more than doing; some actions cannot be undone.
- **Unpredictability**: full consequences cannot be foreseen, compounded by emergent multi-agent effects.

This is why agent safety is not just LLM safety, but more so. Once a model can act within the authorization scope established by delegation, the core issue is no longer only whether an output is wrong, but whether an autonomous system executes beyond what the principal intended to authorize.

1.3 Why This Matters Now

Empirically, the problem is already visible. Recent agent benchmarks and attack studies document indirect prompt injection, memory poisoning, and multi-agent coordination failures in realistic tool-using systems [7, 9, 15, 18, 25, 28, 33, 34, 36]. The point is not merely that open-world delegation creates new attack names. It is that these failures already arise in realistic agent workflows, and that attack-specific or surface-level descriptions do not by themselves explain where the underlying authorization problem entered. Therefore, a useful response must distinguish whether the failure came from ambiguity in what was delegated, corruption through execution channels, or loss of constraints across multi-agent composition, because those cases call for different diagnoses and different defenses.

Existing lines of work are relevant but not sufficient. The paper's claim is that AEG is the right analytical object for understanding a major class of agent safety and security failures, including prompt injection, persistent corruption, and recomposition failure as different structural manifestations of the same underlying problem rather than unrelated attack families. Existing attack and benchmark papers matter here mainly as stakes evidence and observed-failure evidence: they show that the problem is real, but they do not by themselves explain where the underlying authorization problem entered. Threat taxonomies and practitioner guidance [21, 24], and broader frameworks for agent safety and security [1, 3, 14] help organize the literature, but mainly at the level of attacks, controls, or broad system risks rather than at the level of authorization divergence and its structural sources. Classical security abstractions such as reference monitoring, confinement, least privilege, and information-flow control remain relevant [17, 26, 27], but open-world delegated workflows place them in a harder

setting: authorization is partly expressed in natural language, can be reshaped by newly encountered content, and can weaken across memory and multi-agent composition. Alignment methods such as RLHF [23] and Constitutional AI [4] are also relevant, but they mainly seek to improve model alignment as a route to safer behavior. They do not by themselves remove the need to preserve authorization integrity during execution.

For NeurIPS, the paper's contribution is more specific. The argument developed in this paper has three parts: it clarifies how authorization moves from delegation to execution, shows that the same structural sources can be traced across documented agent failures, and derives a source-oriented account of what should be diagnosed, defended against, checked during execution, and reported. If this position is right, agent benchmarks and system papers should not treat task success or isolated attack results as sufficient safety evidence. They should also report how AEG is detected, constrained, and attributed to its structural source during execution.

Our assumptions and research scope. This paper assumes a benign principal and a trusted authorization infrastructure. Within that setting, it studies how realized execution can diverge from the principal's intended task and intended authorization scope in a single delegated workflow or bounded multi-agent system. The analysis excludes three categories of cases: *harmful authorization*, as in explicitly harmful-task settings, such as SAFEARENA [29], AGENTHARM [2], and PENTESTGPT [10]; *compromised authorization infrastructure*, such as tampering with the policy store, provenance substrate, or checking layer, as in TALISMAN [6] and Linux Provenance Modules [5]; and *pure capability failure*, in which authorization remains intact but the agent interprets or plans poorly, or chooses the wrong tool, as in GAIA [19], OSWORLD [30], WEBARENA [37], and ASSISTANTBENCH [32].

2 Delegation, Authorization, and Execution

This section establishes the authorization-execution path from principal intent, through delegation, to realized execution. Each term is introduced at the stage where it enters that path.

2.1 Key Terms

We use the following terms throughout the paper.

Principal, agent, and delegation. The *principal* is the party that delegates a bounded task or role. The *agent* is the system acting on the principal's behalf. *Delegation* is the act by which the principal attempts to convey a bounded mandate to the agent.

Delegated context and authorization relation. *Delegated context* is the concrete signal through which the principal's delegation is conveyed to the agent: the principal's instructions, constraints, clarifications, and other authorization-relevant content that the principal supplies or explicitly endorses. It is the only operationally accessible representation of the principal's intent. The *authorization relation* is the permission relation between principal and agent that is constituted through the agent's receipt of delegated context. It is not an abstract object that exists prior to delegated context.

Intended and interpreted task and authorization scope. The principal's delegation expresses an *intended task* and an *intended authorization scope*: what the principal wants accomplished and the boundary of what the principal means to authorize. Both exist at the level of the principal's intent and are not directly observable by the agent. Upon receiving delegated context, the agent jointly resolves an *interpreted task* and an *interpreted authorization scope*: what it takes the principal to want accomplished, and what it takes the principal to have authorized. These are the agent's operational targets and may diverge from the principal's intended task and intended authorization scope.

Delegated authority and environmental content. An item carries *delegated authority* when it originates from or is explicitly endorsed by the principal. *Environmental content* is material produced or retrieved during execution: tool outputs, web content, stored state, and the outputs of upstream agents. Environmental content may influence later decisions but does not automatically carry delegated authority.

2.2 The Authorization-Execution Path

We describe the authorization-execution path as a sequence of nodes and edges, illustrated in Figure 1, where nodes represent states and edges represent transitions. The structural sources of AEG introduced in later sections map to edges rather than nodes. In the following, we elaborate the path based on each edge.

Edge 1: Delegation. The input to this edge is the principal's *intended task and intended authorization scope*, which can not be directly observed by the agent. Thus, the principal encodes this intent into the *delegated context*, which encompasses the principal's instructions and constraints, together with system-level context such as tool descriptions and operating constraints.

Edge 2: Interpretation. The input to this edge is delegated context. The agent parses the principal's instructions to interpret the intended task and derives the authorization scope from the stated constraints. Where the delegated context is silent, the agent fills in implicit conditions, such as default assumptions about acceptable actions, inferred stopping rules, and background constraints that later execution requires. The output is an *interpreted task* and an *interpreted authorization scope*, and this transition constitutes the authorization relation between principal and agent.

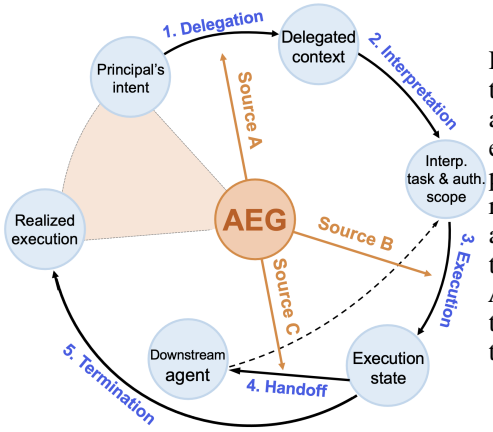

Figure 1: Illustration of the authorization-execution path from principal intent to realized execution, and the three structural sources of AEG mapped to the edges on which they appear.

Edge 3: Execution. The input to this edge is the interpreted task and authorization scope. Before acting, the agent plans a sequence of steps to accomplish the interpreted task within the interpreted authorization scope. It then acts through tool calls, memory reads and writes, and state updates. The output is the *execution state*: the accumulated result of actions taken and state written. Three authorization-relevant events occur along this edge: environmental content enters the decision path, proposed actions become operative, and transient state is written into persistent storage.

Edge 4: Handoff. The input to this edge is execution state. In multi-agent settings, the current agent passes partial results and intermediate state to a downstream agent. From the downstream agent's perspective, this content is environmental content, not delegated context originating from the principal. The authorization relation does not automatically transfer. The downstream agent resolves a new interpreted task and interpreted authorization scope from what it receives, returning the path to the interpreted task and authorization scope node, now held by the downstream agent.

Edge 5: Termination. The input to this edge is the execution state. Execution ends when the interpreted task is complete, a stopping condition is met, or no further actions are available. The path produces its final node, *i.e.*, *realized execution*, including the actual sequence of actions taken, state changes produced, and outputs delivered. This is the concrete object against which the principal's intended task and intended authorization scope can be compared.

2.3 Alice's Travel-Booking Trace

Alice delegates a travel agent with the instruction: "Book an economy flight from New York to Boston tomorrow morning for under $250."

At **Edge 1 (delegation)**, this instruction becomes the delegated context. It leaves implicit whether the budget is a hard ceiling, whether a connecting flight is acceptable, and what the agent should do if no compliant flight exists.

At **Edge 2 (interpretation)**, the authorization relation is constituted. The agent resolves an interpreted task (find and book a qualifying flight) and an interpreted authorization scope (economy class, New York to Boston, tomorrow morning, under $250).

At **Edge 3 (execution)**, search results and pricing responses enter as environmental content, a booking action becomes operative, and payment details are written into persistent state.

At **Edge 4 (handoff)**, if the booking agent passes control to a payment sub-agent, that sub-agent receives partial state from the booking step. Alice's authorization relation does not automatically carry over, and her original constraints may no longer be fully present in what the payment agent receives.

At **Edge 5 (termination)**, the workflow produces a realized execution: a booked flight, a payment record, and a confirmation delivered to Alice. This is the node against which Alice's intended task and intended authorization scope can be compared.

3 The Authorization-Execution Gap

Section 2 described the authorization-execution path from delegation to realized execution. This section explains where the gap between the principal's intended authorization and realized execution can appear along the authorization-execution path, and identifies the three structural sources that cause that gap.

3.1 Where the Gap Appears

Definition. We define the **Authorization-Execution Gap (AEG)** as the divergence between what a principal intends to authorize and what an open-world agent ultimately executes (see the orange filled sector in Figure 1). In this paper, that divergence is analyzed over the full path from intended task and intended authorization scope to realized execution, rather than through any single visible failure at the end of the workflow. Unauthorized action execution, unauthorized disclosure, and related outcomes are downstream manifestations of this gap.

This gap can originate independently from multiple points along the path. It can affect the task dimension, where the agent's interpreted task diverges from what the principal actually wanted accomplished, or the authorization scope dimension, where the agent's interpreted authorization scope diverges from the limit the principal meant to set. At Edge 1, delegated context may fail to fully capture the principal's intended task or intended authorization scope. At Edge 3, environmental content may redirect execution beyond what the principal authorized. At Edge 4, authorization may fragment as the path crosses agent boundaries. Each source can introduce a gap independently, and when multiple sources act together, each can amplify the effect of the others across both dimensions simultaneously.

3.2 Three Structural Sources of the Gap

Three structural sources generate this gap. Each corresponds to an edge on the authorization-execution path, as illustrated in Figure 1. For each source, we identify the **canonical manifestations**: the characteristic forms in which that source's divergence becomes observable as a failure, as shown in Table 1.

Source A: delegation-level incompleteness. This source acts on Edge 1. Delegated context is an incomplete operational proxy for the principal's intended authorization whenever safe execution depends on qualifications, exceptions, escalation conditions, or stopping rules that are not fully expressed in that context. This produces two canonical manifestations: *task misinterpretation* (the agent's interpreted task diverges from the principal's intended task due to ambiguity, underspecification, or multiple plausible readings of the delegated context) and *scope underspecification* (the intended authorization scope is drawn too loosely, allowing the agent to act beyond the limit the principal meant to set).

In Alice's case, the instruction leaves implicit whether "tomorrow morning" means the earliest available departure or any morning flight, leaving the agent's interpreted task potentially misaligned with Alice's intended task. It also leaves implicit whether "under $250" refers to the listed fare or the final charged amount, and whether the agent should stop and ask before crossing the budget by a small amount, leaving the intended authorization scope underspecified. If no compliant flight exists, the agent may further infer that it should relax the time constraint rather than stop and report back, resolving the ambiguity in a way that diverges from what Alice actually wanted accomplished.

Source B: channel-level corruption. This source acts on Edge 3. Independently of whether delegated context fully captures the principal's intended authorization, environmental content entering the decision path can redirect later steps as if it expressed the principal's delegation. This reflects a failure to maintain the distinction between delegated authority and environmental content established in Section 2.1. It produces two canonical manifestations: *authority hijacking* (environmental content directly redirects control flow as if it carried delegated authority) and *authority promotion* (environmental content is written into persistent state and subsequently treated as if it carried delegated authority, converting a transient redirection into a durable one).

In Alice's case, the booking site may display a banner saying "Recommended upgrade: business class for only $40 more" next to the economy option Alice actually authorized. If the ranking agent treats that banner as a reliable preference signal rather than untrusted environmental content, it may rewrite the candidate ordering and pass a business-class itinerary downstream as the preferred choice. The later booking step then follows a path redirected by environmental content rather than by Alice's delegated authority.

Source C: composition-level fragmentation. This source acts on Edge 4. Each downstream agent receives only a partial view of the task and authorization scope interpreted by its upstream agent. As a result, locally acceptable steps can compose into an outcome that falls outside the overall intended authorization scope established by delegation. Local compliance does not guarantee end-to-end authorization coherence. This source produces two canonical manifestations: *recomposition violation* (locally acceptable outputs compose into an artifact that violates the overall intended authorization scope) and *scope accumulation* (the effective authorization scope expands incrementally across sequential handoffs, each locally acceptable, until the aggregate scope exceeds what was originally authorized).

In Alice's case, the intended authorization scope requires three constraints to hold jointly: depart tomorrow morning, remain in economy class, and keep the total charged amount under $250. A planning agent preserves the time constraint and hands off an 8:00 AM itinerary; a ranking agent preserves the class constraint by selecting an economy ticket listed at $235; a downstream booking service adds $28 in taxes and seat fees while checking only inventory and payment completion, not Alice's original budget constraint. No stage recomposes all three constraints against the final transaction state, and the workflow ends with a final charged amount of $263 that violates Alice's intended authorization scope, though each local step appeared acceptable under its own partial view.

How the sources interact. These sources can amplify one another. Source A may leave the budget condition underspecified, Source B may allow the booking site's upgrade banner to influence ranking as if it expressed Alice's delegated authority, and Source C may carry that distortion across planning, ranking, and booking without any step recomposing the full set of constraints. The workflow can then end in a business-class booking that departs at an acceptable time but falls outside Alice's intended authorization scope, even though each local step appeared justified under its own partial view.

4 Failure Diagnosis and Authorization Integrity Checks During Execution

This section turns the structural analysis of Section 3.2 into its practical consequences. Failures should be diagnosed by structural source rather than by observed failure category alone. We define **authorization integrity** as the property that realized execution conforms to the principal's intended task and intended authorization scope. Because the structural sources act on edges of the authorization-execution path, preserving authorization integrity requires checks at those edges during execution, and reporting that makes those checks and their attribution consequences visible.

4.1 Diagnosing Failures by Structural Source

Why observed failure categories alone are insufficient. The same observed failure category can arise from different structural sources. For example, the observed category *unauthorized information disclosure* does not by itself tell us whether the problem arose because the intended authorization scope was underspecified (Source A), because environmental content redirected the workflow (Source B), or because locally acceptable steps composed into a globally unauthorized outcome (Source C). Table 1 illustrates this point across representative failure categories. Diagnosis that stops at the observed

Table 1: Expanded mapping from observed failure categories to representative documented failure or attack patterns, their canonical manifestations (defined in Section 3.2), and their primary structural sources. Pattern labels in the second column are representative names used by this paper; citations indicate documented exemplars rather than exact terminology matches.

Observed failure category	**Representative documented failure / attack pattern**	**Canonical manifestation**	**Primary source**
Unauthorized information disclosure	Over-broad retrieval or disclosure from underspecified delegated context [13, 35]	Scope underspecification	A
	Prompt- or tool-output-induced data exfiltration [9, 33]	Authority hijacking	B
	Cross-agent or cross-stage leakage after locally acceptable handoffs [12, 22]	Recomposition violation	C
Unauthorized action execution	Over-execution or misdirected execution under ambiguous or underspecified task constraints [8, 16]	Task misinterpretation	A
	Prompt-, tool-, or interface-induced unintended action [15, 33]	Authority hijacking	B
	Multi-stage or multi-agent completion outside the overall intended authorization scope [18, 20, 36]	Recomposition violation	C
Persistent state corruption	Incorrect durable update under underspecified write conditions [8, 16]	Scope underspecification	A
	Untrusted content promoted into memory or persistent state [7, 11, 31]	Authority promotion	B
	Persistent-state corruption propagated across tools, agents, or workflow stages [7, 12, 36]	Scope accumulation	C

failure category cannot identify which edge of the authorization-execution path the divergence entered from, and therefore cannot identify the appropriate defense.

Diagnosis by structural source. The right unit of diagnosis is structural source: which edge the divergence entered from, and how that divergence produced the concrete failure. Table 1 supports this by mapping representative failure categories to canonical manifestations and their primary structural sources. Once diagnosis is organized this way, defense and intervention design can target the mechanism by which the divergence was introduced rather than patching one visible failure category at a time. It also makes failure analysis more inspectable: investigators can ask which structural source introduced or widened the divergence, and at which edge a relevant check was missing, insufficient, or bypassed.

4.2 Why Authorization Integrity Must Be Checked During Execution

One-shot checking is insufficient. A check applied only at delegation time cannot intercept the divergence that the three structural sources generate. Source A acts on Edge 1, but its effects may not become operative until Edge 2 or Edge 3. Sources B and C act on Edges 3 and 4 respectively, during execution itself. None of these events are fully visible in the initial delegated context, so no upfront check can anticipate them.

Execution-time checks are therefore necessary. Safety training, safer tools, and tighter workflow design can narrow the conditions under which divergence occurs, but they do not guarantee authorization integrity at the edges where environmental content enters the decision path, proposed actions become operative, state is written, and outputs are handed off. In open-world, tool-using, or multi-agent settings, those edges remain live regardless of how carefully the system is designed upfront. Section 4.3 specifies what checks are needed at each edge.

4.3 Authorization Integrity Checks

The five checks below map directly to the authorization-execution path in Section 2.2. Each is tied to an edge and asks whether divergence should be stopped before it becomes operative, durable, or exportable to a downstream agent.

Delegation Completeness Check. This check intervenes at the entry of Edge 2, before *interpretation* proceeds, and targets the *delegation-level incompleteness* introduced by Source A. At the edge from delegated context to interpreted task and authorization scope, check whether unresolved qualifications, exceptions, escalation conditions, or stopping rules are present before interpretation proceeds. If so, surface them before reasoning or action planning treats them as already resolved. Incompleteness in delegated context can otherwise be silently converted into an operative authorization assumption at this edge.

Authority Attribution Check. This check intervenes at the *execution edge* ($i.e.$, Edge 3) and targets the *channel-level corruption* introduced by Source B. At the point where environmental content enters the decision path, check whether each item is correctly attributed as delegated authority or environmental content. Environmental content misattributed as delegated authority can redirect downstream decisions without any change to the delegated context itself.

Scope Compliance Check. This check intervenes at the *execution edge* ($i.e.$, Edge 3) and targets the *channel-level corruption* introduced by Source B, and the *composition-level fragmentation* introduced by Source C. At the point where a proposed action becomes operative, check whether the proposed step falls within the interpreted authorization scope established at Edge 2. Divergence that was previously latent becomes operative and potentially irreversible at this point.

Provenance Preservation Check. This check intervenes at the *execution edge* ($i.e.$, Edge 3) and targets the *channel-level corruption* introduced by Source B. At the point where transient state is written into persistent storage, check that provenance is preserved and that environmental content is not written in forms that later appear to carry delegated authority. A transient redirection can otherwise become durable and propagate across later edges.

Recomposition Authorization Check. This check intervenes at the *handoff edge* ($i.e.$, Edge 4) and targets the *composition-level fragmentation* introduced by Source C. It checks whether the fragments being transmitted to the downstream agent remain within the interpreted authorization scope established at Edge 2 when recomposed. The authorization relation does not automatically transfer through a handoff, and locally acceptable fragments can compose into a globally unauthorized outcome.

These checks are not a full defense architecture. Their role is narrower: to make explicit where authorization integrity must be preserved during execution. The thesis does not require perfect recovery of the principal's intent. It requires enough edge-local evidence to decide whether execution should proceed, pause, or escalate, and enough discipline not to silently resolve uncertainty in favor of action.

4.4 What This Implies for Reporting and Evaluation

Current evidence is insufficient. Aggregate capability, task success, and attack-specific evidence can all miss whether the agent stayed within the principal's intended authorization scope while acting. A system may succeed on the task as interpreted while still allowing environmental content to redirect the decision path at Edge 3, allowing state written at Edge 3 to later be treated as delegated authority, or allowing fragments transmitted at Edge 4 to compose into an outcome outside the intended authorization scope. Under the position advanced here, those failures matter even when the visible task outcome looks successful [9, 25, 28, 34].

Source attribution is also missing. The problem is not only that current evidence can miss such failures, but also that it often does not show which structural source was responsible or at which edge a relevant check was absent. A paper may report strong task completion or a handful of isolated attack results while leaving unclear whether a gap was prevented at the relevant edges, merely observed after the fact, or never attributed to a structural source.

A lightweight reporting step. One concrete community step would be a short authorization-safety report attached to agent papers for systems with meaningful delegated autonomy across tools, state, or sub-agents. At minimum, it should indicate which of the five checks described in Section 4.3 are present, which are absent, and what evidence was collected at those edges and the termination node. When evidence is available, it should also indicate where observed divergence was attributed to a structural source and at which edge it originated. This is a minimal reporting implication rather than a demand for one fixed benchmark or defense architecture, and is closer to a lightweight risk-reporting supplement than to a new benchmark track [21, 24].

5 Objections

Objection 1: "Is this only a new label for known failures?" Response: The paper does not claim to discover a new attack family or a new observed failure category. Its claim is that many known failures become clearer when they are traced to structural source rather than described only at the level of observed failure category. That reorganization matters for two reasons. First, it supports better defense design: the same observed failure category can arise from different structural sources, so a defense aimed only at the surface pattern may miss the cause. Second, it supports better attribution: when a failure is traced to delegation-level incompleteness, channel-level corruption, or composition-level fragmentation, the responsibility can be assigned more precisely to the delegation act, the execution channel, or the composition boundary at which the gap originated.

Objection 2: "Why not rely mainly on constrained workflows and human approval checkpoints?" Response: Constrained workflows, capability bounds, and human approval checkpoints are important controls. For many systems they may reduce risk substantially, and human approval checkpoints can themselves serve as authorization-integrity checks during execution. The paper's claim is narrower: these controls do not by themselves guarantee that later execution remains within the authorization scope established by delegation once environmental content enters the decision path, persistent state is updated, or work is handed off downstream. In more open delegated workflows, a system can satisfy early constraints and still diverge later in execution. Source-oriented authorization-integrity checks during execution are therefore complements to constrained design and approval checkpoints, not replacements for them [9, 33, 34].

Objection 3: "How is this different from generic runtime monitoring?" Response: Generic runtime monitoring asks whether a system shows anomalies, unsafe outputs, or policy violations, but typically does not identify which structural source produced them or at which edge they originated. Source-oriented authorization-integrity checks make that source and origin explicit, which is what defense design and failure attribution require. The paper therefore argues not for a generic monitoring layer, but for checks grounded in the structural sources of authorization divergence.

Together, these objections clarify the boundaries of the thesis without overturning its core logic. AEG is not proposed as a replacement for all agent security and safety analysis; it is proposed as a structural lens for failures that the observed failure category alone cannot adequately diagnose in open-world delegated systems.

6 Conclusion

For open-world, tool-using, stateful, or multi-agent systems, the Authorization-Execution Gap is a major safety and security problem. Realized execution can diverge from the principal's intended authorization through three structural sources: delegation-level incompleteness, channel-level corruption, and composition-level fragmentation. Many observed failures become clearer when traced to these sources rather than treated as isolated attack types. Because the same observed failure can arise from different sources, diagnosis and defense should be source-oriented. Because these sources act on edges of the authorization-execution path, safety and security require authorization-integrity checks during execution, not only upfront filtering or post-hoc audit. For NeurIPS, the implication is that papers on open-world agents should report not only capability, task success, or attack outcomes, but also process-level evidence showing where divergence was detected, constrained, and attributed to a structural source during execution.

References

[1] Edoardo Allegrini, Ananth Shreekumar, and Z. Berkay Celik. Formalizing the safety, security, and functional properties of agentic ai systems. In *ICLR 2026 Workshop on Agentic AI for the Real World: Risks, Safety, and Responsible Innovation*, 2026.

[2] Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, Zico Kolter, Matt Fredrikson, Eric Winsor, Jerome Wynne, Yarin Gal, and Xander Davies. Agentharm: A benchmark for measuring harmfulness of llm agents. In *International Conference on Learning Representations*, 2025.

[3] Sunil Arora and John Hastings. Securing agentic ai systems—a multilayer security framework. *arXiv preprint arXiv:2512.18043*, 2025.

[4] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. Constitutional ai: Harmlessness from ai feedback. *arXiv preprint arXiv:2212.08073*, 2022.

[5] Adam Bates, Dave (Jing) Tian, Kevin R. B. Butler, and Thomas Moyer. Trustworthy whole-system provenance for the linux kernel. In *24th USENIX Security Symposium*, 2015.

[6] Frank Capobianco, Quan Zhou, Aditya Basu, Trent Jaeger, and Danfeng Zhang. Talisman: Tamper analysis for reference monitors. In *Network and Distributed System Security Symposium (NDSS)*, 2024.

[7] Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. Agentpoison: Red-teaming llm agents via poisoning memory or knowledge bases. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[8] Pengzhou Cheng, Zheng Wu, Zongru Wu, Ju Tianjie, Aston Zhang, Zhuosheng Zhang, and Gongshen Liu. Os-kairos: Adaptive interaction for mllm-powered gui agents. In *Findings of Proceedings of the 63th Annual Meeting of the Association for Computational Linguistics*, 2025.

[9] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[10] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. Pentestgpt: An llm-empowered automatic penetration testing tool. *arXiv preprint arXiv:2308.06782*, 2023.

[11] Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, Tianming Liu, Hui Liu, and Zhen Xiang. Memory injection attacks on llm agents via query-only interaction. *arXiv preprint arXiv:2503.03704*, 2025.

[12] Faouzi El Yagoubi, Ranwa Al Mallah, and Godwin Badu-Marfo. Agentleak: A full-stack benchmark for privacy leakage in multi-agent llm systems. *arXiv preprint arXiv:2602.11510*, 2026.

[13] Wenjie Fu, Xiaoting Qin, Jue Zhang, Qingwei Lin, Lukas Wutschitz, Robert Sim, Saravan Rajmohan, and Dongmei Zhang. Ci-work: Benchmarking contextual integrity in enterprise llm agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics: Industry Track*, 2026.

[14] Shaona Ghosh, Barnaby Simkin, Kyriacos Shiarlis, Soumili Nandi, Dan Zhao, Matthew Fiedler, Julia Bazinska, Nikki Pope, Roopa Prabhu, Daniel Rohrer, Michael Demoret, and Bartley Richardson. A safety and security framework for real-world agentic systems. *arXiv preprint arXiv:2511.21990*, 2025.

[15] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, 2023.

[16] Jaylen Jones, Zhehao Zhang, Yuting Ning, Eric Fosler-Lussier, Pierre-Luc St-Charles, Yoshua Bengio, Dawn Song, Yu Su, and Huan Sun. When benign inputs lead to severe harms: Eliciting unsafe unintended behaviors of computer-use agents. *arXiv preprint arXiv:2602.08235*, 2026.

[17] Butler W. Lampson. A note on the confinement problem. *Communications of the ACM*, 16(10): 613–615, 1973.

[18] Yihuan Mao, Yipeng Kang, Peilun Li, Wei Xu, and Chongjie Zhang. Ibgp: Imperfect byzantine generals problem for zero-shot robustness in communicative multi-agent systems. In *Artificial General Intelligence*, volume 16057 of *Lecture Notes in Computer Science*. Springer, 2025.

[19] Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. Gaia: a benchmark for general ai assistants. In *International Conference on Learning Representations*, 2024.

[20] Yifei Ming, Zixuan Ke, Xuan-Phi Nguyen, Jiayu Wang, and Shafiq Joty. Helpful agent meets deceptive judge: Understanding vulnerabilities in agentic workflows. *arXiv preprint arXiv:2506.03332*, 2025.

[21] MITRE. Mitre atlas: Adversarial threat landscape for ai systems, 2025.

[22] Ivoline C. Ngong, Keerthiram Murugesan, Swanand Kadhe, Justin D. Weisz, Amit Dhurandhar, and Karthikeyan Natesan Ramamurthy. Agentscope: Evaluating contextual privacy across agentic workflows. *arXiv preprint arXiv:2603.04902*, 2026.

[23] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

[24] OWASP Foundation. Owasp agentic security guidance, 2025.

[25] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J Maddison, and Tatsunori Hashimoto. Identifying the risks of lm agents with an lm-emulated sandbox. In *The Twelfth International Conference on Learning Representations*, 2024.

[26] Andrei Sabelfeld and Andrew C. Myers. Language-based information-flow security. *IEEE Journal on Selected Areas in Communications*, 21(1):5–19, 2003.

[27] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.

[28] Natalie Shapira, Chris Wendler, Avery Yen, Gabriele Sarti, Koyena Pal, Olivia Floody, Adam Belfki, Alex Loftus, Aditya Ratan Jannali, Nikhil Prakash, Jasmine Cui, Giordano Rogers, Jannik Brinkmann, Can Rager, Amir Zur, Michael Ripa, Aruna Sankaranarayanan, David Atkinson, Rohit Gandikota, Jaden Fiotto-Kaufman, EunJeong Hwang, Hadas Orgad, P Sam Sahil, Negev Taglicht, Tomer Shabtay, Atai Ambus, Nitay Alon, Shiri Oron, Ayelet Gordon-Tapiero, Yotam Kaplan, Vered Shwartz, Tamar Rott Shaham, Christoph Riedl, Reuth Mirsky, Maarten Sap, David Manheim, Tomer Ullman, and David Bau. Agents of chaos. *arXiv preprint arXiv:2602.20021*, 2026.

[29] Ada Defne Tur, Nicholas Meade, Xing Han Lù, Alejandra Zambrano, Arkil Patel, Esin Durmus, Spandana Gella, Karolina Stańczak, and Siva Reddy. Safearena: Evaluating the safety of autonomous web agents. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025.

[30] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[31] Xianglin Yang, Yufei He, Shuo Ji, Bryan Hooi, and Jin Song Dong. Zombie agents: Persistent control of self-evolving llm agents via self-reinforcing injections. *arXiv preprint arXiv:2602.15654*, 2026.

[32] Ori Yoran, Samuel Joseph Amouyal, Chaitanya Malaviya, Ben Bogin, Ofir Press, and Jonathan Berant. Assistantbench: Can web agents solve realistic and time-consuming tasks? In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, 2024.

[33] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of Proceedings of the 62th Annual Meeting of the Association for Computational Linguistics*, 2024.

[34] Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. Agent security bench (asb): Formalizing and benchmarking attacks and defenses in llm-based agents. In *The Thirteenth International Conference on Learning Representations*, 2025.

[35] Arman Zharmagambetov, Chuan Guo, Ivan Evtimov, Maya Pavlova, Ruslan Salakhutdinov, and Kamalika Chaudhuri. Agentdam: Privacy leakage evaluation for autonomous web agents. *arXiv preprint arXiv:2503.09780*, 2025.

[36] Lifan Zheng, Jiawei Chen, Qinghong Yin, Jingyuan Zhang, Xinyi Zeng, and Yu Tian. Rethinking the reliability of multi-agent system: A perspective from byzantine fault tolerance. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 2026.

[37] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2024.